

Module 8) Laravel Framework

Theory Assignments

1. Introduction to Laravel

o Assignment: Write a detailed report on the history of Laravel. Include its versioning, key features, and how it differs from other PHP frameworks.

History of Laravel

Laravel was developed by Taylor Otwell and officially released in **June 2011**. It was created to overcome limitations in existing PHP frameworks like CodeIgniter, especially the lack of built-in features such as authentication and advanced routing.

Evolution of Laravel Versions

- **Laravel 1 (2011)**
The first version introduced basic routing, authentication, and views. However, it did not follow the MVC pattern.
- **Laravel 2 (2011)**
Added support for MVC architecture, making development more structured.
- **Laravel 3 (2012)**
Introduced the Artisan command-line tool, database migrations, and Eloquent ORM.
- **Laravel 4 (2013)**
Completely redesigned framework using Composer. Introduced modular packaging system.
- **Laravel 5 (2015)**
Major improvements including middleware, task scheduling, and improved authentication.
- **Laravel 6 (2019 - LTS)**
First version to adopt semantic versioning. Long-term support version.
- **Laravel 7 (2020)**
Introduced HTTP client, improved routing, and Blade components.
- **Laravel 8 (2020)**
Added features like job batching and Laravel Jetstream.
- **Laravel 9 (2022)**
Updated dependencies like Symfony and improved performance.
- **Laravel 10 (2023)**
Focused on code quality, performance, and developer experience.

- **Laravel 11 (2024+)**
Simplified project structure and reduced boilerplate code.

Laravel Versioning System

Laravel follows **Semantic Versioning (SemVer)**:

Format:

MAJOR.MINOR.PATCH (Example: 10.5.2)

Explanation:

- **Major Version:** Includes breaking changes
- **Minor Version:** Adds new features without breaking existing code
- **Patch Version:** Fixes bugs and improves stability

LTS (Long-Term Support) Versions:

- Bug fixes for **2 years**
- Security fixes for **3 years**

This versioning system ensures stability while allowing continuous improvement.

Key Features of Laravel

1. MVC Architecture

Separates application logic into Model, View, and Controller, improving code organization.

2. Blade Templating Engine

Laravel provides a lightweight yet powerful templating engine for building dynamic web pages.

3. Eloquent ORM

An Object-Relational Mapping system that allows developers to interact with databases using PHP syntax instead of SQL.

4. Artisan CLI

A command-line interface that automates tasks like migrations, controller creation, and testing.

5. Routing System

Laravel offers a simple and expressive routing system that supports RESTful APIs.

6. Middleware

Filters HTTP requests for authentication, logging, and security.

7. Built-in Authentication

Provides ready-to-use authentication and authorization systems.

8. Security Features

Protects applications from:

- SQL Injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)

9. Database Migration

Allows version control of database structure.

10. Task Scheduling

Simplifies cron job management using a clean syntax.

11. Queue System

Handles background jobs such as sending emails and processing tasks.

Difference Between Laravel and Other PHP Frameworks

Laravel vs CodeIgniter

- Laravel provides built-in tools like ORM and authentication
- CodeIgniter is lightweight but requires manual implementation

Laravel vs Symfony

- Symfony is more complex and suited for large enterprise applications
- Laravel is easier to learn and faster for development

Laravel vs CakePHP

- Laravel offers better modern features and flexibility
- CakePHP follows strict conventions but has a smaller ecosystem

2. Laravel MVC Architecture

o Assignment: Explain the MVC (Model-View-Controller) architecture. Provide examples of how Laravel implements this architecture in web applications.

MVC (Model-View-Controller) is a software design pattern used to organize code in web applications. It separates the application into three main components:

- **Model** → Handles data and database logic
- **View** → Handles user interface (UI)
- **Controller** → Handles request logic and connects Model & View

This separation makes applications easier to manage, scale, and debug.

Laravel MVC Example (Complete Flow)

Step 1: Route

```
Route::get('/user/{id}', [UserController::class, 'show']);
```

Step 2: Controller

```
public function show($id)
{
    $user = User::find($id);
    return view('user', compact('user'));
}
```

Step 3: Model

```
$user = User::find($id);
```

Step 4: View

```
<h1>{{ $user->name }}</h1>
```

3. Routing in Laravel

o Assignment: Describe how routing works in Laravel. Explain the difference between named routes and route parameters with examples.

Routing in Laravel defines how an application responds to a user request (URL). It maps a URL to a specific function, controller, or action.

Example:

```
Route::get('/home', function () {
    return "Welcome to Home Page";
});
```

When a user visits /home, Laravel executes the defined function and returns the response.

How Routing Works in Laravel

Step-by-Step Flow:

1. User enters a URL in the browser
2. Laravel checks routes defined in routes/web.php
3. Matching route is found
4. Route calls a function or controller
5. Response is returned to the user

Named Routes vs Route Parameters

Feature	Named Routes	Route Parameters
Purpose	Identify routes with a name	Pass dynamic data in URL
Usage	<code>route('profile')</code>	<code>/user/{id}</code>
Example	<code>->name('home')</code>	<code>{id}</code>
Benefit	Easy URL management	Dynamic content handling

4. Blade Templating Engine

o Assignment: Write an essay on the Blade templating engine in Laravel. Discuss its features, syntax, and how it enhances the development process.

Blade is the powerful and lightweight templating engine provided by Laravel. It allows developers to create dynamic and reusable views with clean and readable syntax. Unlike traditional PHP templating, Blade simplifies the process of embedding PHP code within HTML, making web development faster and more efficient.

What is Blade Templating Engine?

Blade is Laravel's built-in templating engine that helps in separating the presentation layer from the business logic. It enables developers to write templates using simple directives instead of complex PHP code.

Blade files use the .blade.php extension and are stored in the resources/views directory.

Features of Blade

1. Simple and Clean Syntax

Blade uses an easy-to-understand syntax that reduces the need for writing raw PHP code.

```
{{ $name }}
```

This automatically echoes and escapes the variable.

2. Template Inheritance

Blade supports layout inheritance, allowing developers to reuse common layouts.

```
<!-- layout.blade.php -->
<html>
<body>
    @yield('content')
</body>
</html>

<!-- child.blade.php -->
@extends('layout')

@section('content')
    <h1>Welcome</h1>
@endsection
```

3. Blade Directives

Blade provides built-in directives for common tasks:

- @if, @else, @endif
- @foreach, @endforeach
- @include
- @extends

Example:

```
@if($user)
    Welcome {{ $user->name }}
@endif
```

4. Components and Slots

Blade allows reusable components for UI elements.

```
<x-alert type="success" message="Saved successfully!" />
```

5. Automatic Escaping

Blade protects against XSS attacks by escaping output automatically:

```
{{ $data }}
```

To render raw HTML:

```
{!! $data !!}
```

6. Control Structures

Blade simplifies loops and conditions:

```
@foreach($users as $user)
    <p>{{ $user->name }}</p>
@endforeach
```

7. Includes and Partial

You can reuse parts of views:

```
@include('header')
@include('footer')
```

Blade Syntax Overview

Feature	Syntax Example
Echo Data	{{ \$var }}
Raw Output	{!! \$var !!}
If Condition	@if(condition)
Loop	@foreach
Extend Layout	@extends('layout')
Section	@section('content')

How Blade Enhances Development Process

1. Improves Code Readability

Blade syntax is clean and easy to understand, making code more readable than plain PHP.

2. Encourages Code Reusability

With layouts, components, and includes, developers can reuse code and avoid duplication.

3. Faster Development

Blade reduces the amount of code needed, speeding up development time.

4. Better Security

Automatic escaping protects against common vulnerabilities like XSS attacks.

5. Separation of Concerns

Blade ensures that UI (views) is separate from logic (controllers and models), following MVC principles.

6. Easy Maintenance

Changes in layout or components automatically reflect across the application.

5. Database Migrations and Eloquent ORM

o Assignment: Explain the concept of database migrations in Laravel. Discuss how Eloquent ORM simplifies database interactions and provide examples of CRUD operations.

Database Migrations and Eloquent ORM in Laravel

Introduction

Laravel provides powerful tools to manage databases efficiently. Two of the most important features are **Database Migrations** and **Eloquent ORM**. Migrations help manage database structure, while Eloquent ORM simplifies database interactions using object-oriented syntax.

What are Database Migrations?

Database migrations in Laravel act like **version control for your database**. Instead of manually creating tables in phpMyAdmin, you define them using PHP code.

Key Benefits:

- Track database changes over time
- Easily share database structure with team
- Rollback changes if needed
- Maintain consistency across environments

Migrations Work :

Migrations are stored in the database/migrations folder and are executed using Artisan commands.

Create Migration:

```
php artisan make:migration create_users_table
```

Example Migration File:

```
public function up()
{
    Schema::create('users', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email')->unique();
        $table->timestamps();
    });
}

public function down()
{
    Schema::dropIfExists('users');
}
```

Run Migration:

```
php artisan migrate
```

Eloquent ORM :

Eloquent ORM (Object-Relational Mapping) is Laravel's built-in tool that allows developers to interact with the database using PHP objects instead of writing raw SQL queries.

Example:

```
User::all();
```

Eloquent Simplifies Database Interaction :

1. Object-Oriented Syntax

Work with database tables as models.

2. Automatic Table Mapping

Model name automatically maps to table name.

3. Easy Relationships

Supports relationships like:

- One-to-One

- One-to-Many
- Many-to-Many

4. Built-in Methods

Predefined methods for common operations like find(), save(), delete().

CRUD Operations Using Eloquent ORM :

CRUD stands for:

- Create
- Read
- Update
- Delete

1. Create (Insert Data)

```
use App\Models\User;
$user = new User();
$user->name = "Riddhi";
$user->email = "riddhi@gmail.com";
$user->save();
```

👉 OR using mass assignment:

```
User::create([
    'name' => 'Riddhi',
    'email' => 'riddhi@gmail.com'
]);
```

2. Read (Fetch Data)

```
$users = User::all(); // get all users
```

```
$user = User::find(1); // find by ID
```

3. Update (Modify Data)

```
$user = User::find(1);
$user->name = "Updated Name";
$user->save();
```

4. Delete (Remove Data)

```
$user = User::find(1);
$user->delete();
```

6. Laravel Middleware

o Assignment: Define middleware in Laravel. Explain how middleware can be used for authentication, logging, and CORS handling.

Definition of Middleware

Middleware in Laravel acts as a **filter for HTTP requests** entering your application. It sits between the request and the application logic, allowing you to inspect, modify, or reject requests before they reach the controller.

Middleware for Authentication

Authentication middleware ensures only logged-in users can access certain pages.

Example:

```
Route::get('/dashboard', function () {  
    return "Welcome to Dashboard";  
})->middleware('auth');
```

If user is not logged in → redirected to login page

Middleware for Logging

Middleware can log request details such as URL, IP, and time.

Example:

```
public function handle($request, Closure $next)  
{  
    \Log::info('User accessed: ' . $request->path());  
  
    return $next($request);  
}
```

This helps in:

- Debugging
- Monitoring user activity
- Tracking errors

Middleware for CORS Handling

CORS (Cross-Origin Resource Sharing) controls access between different domains.

Example:

```

public function handle($request, Closure $next)
{
    return $next($request)
        ->header('Access-Control-Allow-Origin', '*')
        ->header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE')
        ->header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
}

```

This allows APIs to be accessed from different domains.

7. Laravel Authentication

o Assignment: Write a report on Laravel's built-in authentication system.

Explain how to set up user authentication and discuss the use of guards and providers.

Authentication is the process of verifying a user's identity before allowing access to a system. Laravel provides a robust and secure built-in authentication system that simplifies the implementation of login, registration, password reset, and user management features.

With minimal configuration, developers can quickly set up authentication and protect application routes, making Laravel highly efficient for modern web development.

Laravel offers ready-made solutions for authentication through starter kits such as:

- **Laravel Breeze** – Simple and lightweight
- **Laravel Jetstream** – Advanced features (teams, API tokens)
- **Laravel UI** – Basic authentication scaffolding

These tools generate controllers, views, and routes required for authentication, reducing development time.

Steps to Set Up User Authentication

Step 1: Install Laravel Project

```
composer create-project laravel/laravel project-name
```

Step 2: Install Laravel Breeze

```
composer require laravel/breeze --dev
php artisan breeze:install
```

Step 3: Install Frontend Dependencies

```
npm install  
npm run dev
```

Step 4: Run Database Migrations

```
php artisan migrate
```

This creates tables such as:

- users
- password_resets

Step 5: Run Application

```
php artisan serve
```

8. Testing in Laravel

o Assignment: Discuss the importance of testing in web applications. Explain the testing tools available in Laravel and write a brief guide on how to write basic tests.

Testing is a crucial part of web application development. It ensures that the application works correctly, is free from bugs, and meets user requirements. In modern development, testing helps maintain code quality, improve performance, and prevent future errors.

Laravel provides built-in support for testing, making it easier for developers to write and run tests efficiently.

Importance of Testing in Web Applications :

1. Ensures Code Quality

Testing verifies that each part of the application functions as expected.

2. Detects Bugs Early

Finding bugs early reduces development cost and effort.

3. Improves Security

Testing helps identify vulnerabilities in authentication, validation, and APIs.

4. Enhances Performance

Performance testing ensures the application handles load efficiently.

5. Supports Maintenance

With tests in place, developers can update code without breaking existing features.

6. Builds Confidence

Developers can deploy applications confidently knowing the system is tested.

Testing Tools in Laravel

Laravel provides several built-in tools and integrations for testing:

1. PHPUnit

- Default testing framework in Laravel
- Used for writing unit and feature tests

Example command:

```
php artisan test
```

2. Laravel Test Helpers

Laravel provides helper methods for testing routes, responses, and database.

Examples:

- `get()` → Test GET request
- `post()` → Test POST request
- `assertStatus()` → Check HTTP status

3. Feature Testing

Tests complete application features like routes, controllers, and database.

4. Unit Testing

Tests individual components like functions or classes.

5. Database Testing

- `RefreshDatabase` trait resets database after each test
- Ensures clean test environment

6. Mocking & Factories

- Factories generate dummy data
- Mocking simulates dependencies

Practical Assignments

1. Setting Up a Laravel Project

o Task: Install Laravel using Composer and create a new Laravel project. Set up your development environment (including a local server) and configure the .env file for database connections.

1. Prerequisites

Before installing Laravel, make sure you have:

- **PHP (>= 8.1)**
- **Composer**
- **Node.js & npm**
- **Database (MySQL / MariaDB)**
- **Local server like XAMPP / WAMP / Laragon**

2. Install Composer

Composer is required to install Laravel.

Download from: [Composer](#)

Check Installation:

```
composer -v
```

3. Install Laravel

Method 1: Using Composer (Recommended)

```
composer create-project laravel/laravel myProject
```

This creates a new Laravel project folder named myProject.

Method 2: Using Laravel Installer (Optional)

```
composer global require laravel/installer  
laravel new myProject
```

4. Navigate to Project Folder

```
cd myProject
```

5. Start Local Development Server

Laravel provides a built-in server:

```
php artisan serve
```

👉 Open in browser:

`http://127.0.0.1:8000`

6. Set Up Database

Step 1: Create Database

Open **phpMyAdmin** (via XAMPP)

Create a database, for example:

`myproject_db`

7. Configure .env File

The .env file stores environment settings like database credentials.

👉 Open .env file and update:

`DB_CONNECTION=mysql`

`DB_HOST=127.0.0.1`

`DB_PORT=3306`

`DB_DATABASE=myproject_db`

`DB_USERNAME=root`

`DB_PASSWORD=`

8. Run Migrations

This will create default tables like users.

`php artisan migrate`

2. Creating a Simple Blog

o Task: Develop a simple blog application where users can create, read, update, and delete (CRUD) blog posts.

o Requirements:

- **Use Eloquent ORM for database operations.**
- **Implement route controllers for handling requests.**
- **Use Blade templates for the front end.**

Step 1: Create Model, Controller & Migration

`php artisan make:model Post -mcr`

Step 2: Define Routes

Open routes/web.php

```
use App\Http\Controllers\PostController;
```

```
Route::view('posts', PostController::class, 'index');
```

Step 3: Controller Logic (CRUD)

app/Http/Controllers/PostController.php

app/Models/Post.php

```
public function index()
{
    $posts = Post::all();
    return view('posts.index', compact('posts'));
}

public function create()
{
    return view('posts.create');
}

public function store(Request $request)
{
    Post::create($request->all());
    return redirect()->route('posts.index');
}

public function show(Post $post)
{
    return view('posts.show', compact('post'));
}

public function edit(Post $post)
{
    return view('posts.edit', compact('post'));
}

public function update(Request $request, Post $post)
{
    $post->update($request->all());
    return redirect()->route('posts.index');
}

public function destroy(Post $post)
{
    $post->delete();
}
```

```
    return redirect()->route('posts.index');
}
```

Blade Templates :

1. Layout (resources/views/layout.blade.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Blog</title>
</head>
<body>
    <h1>Simple Blog</h1>
    @yield('content')
</body>
</html>
```

2. index.blade.php

```
@extends('layout')
@section('content')
<a href="{{ route('posts.create') }}">Create Post</a>
@foreach($posts as $post)
    <h2>{{ $post->title }}</h2>
    <p>{{ $post->content }}</p>
    <a href="{{ route('posts.show', $post->id) }}">View</a>
    <a href="{{ route('posts.edit', $post->id) }}">Edit</a>
    <form action="{{ route('posts.destroy', $post->id) }}" method="POST">
        @csrf
        @method('DELETE')
        <button type="submit">Delete</button>
    </form>
@endforeach
@endsection
```

Create Page

```
@extends('layout')

@section('content')
<form action="{{ route('posts.store') }}" method="POST">
    @csrf
    <input type="text" name="title" placeholder="Title"><br>
```

```

        <textarea name="content" placeholder="Content"></textarea><br>
        <button type="submit">Save</button>
    </form>
@endsection

```

Edit Page

```

@extends('layout')

@section('content')
<form action="{{ route('posts.update', $post->id) }}" method="POST">
    @csrf
    @method('PUT')

    <input type="text" name="title" value="{{ $post->title }}"><br>
    <textarea name="content">{{ $post->content }}</textarea><br>
    <button type="submit">Update</button>
</form>
@endsection

```

Show Page

```

@extends('layout')

@section('content')
<h2>{{ $post->title }}</h2>
<p>{{ $post->content }}</p>

<a href="{{ route('posts.index') }}">Back</a>
@endsection

```

3. User Registration and Authentication

o Task: Implement a user registration and login system using Laravel's built-in authentication.

o Requirements:

- Allow users to register with a username, email, and password.
- Implement email verification.
- Create a dashboard that displays user information after login

create_users_table :

```
$table->string('name'); // username
```

```
$table->string('email')->unique();
```

```
$table->string('password');
```

```
resources/views/auth/register.blade.php
```

```
<input type="text" name="name" placeholder="Username" required>
```

```
<input type="email" name="email" required>
```

```
<input type="password" name="password" required>
```

Configure Mail (for verification)

```
.env
```

```
MAIL_MAILER=smtp
```

```
MAIL_HOST=smtp.mailtrap.io
```

```
MAIL_PORT=2525
```

```
MAIL_USERNAME=your_username
```

```
MAIL_PASSWORD=your_password
```

```
MAIL_ENCRYPTION=null
```

```
resources/views/dashboard.blade.php
```

```
@extends('layout')
```

```
@section('content')
```

```
<h1>Welcome, {{ auth()->user()->name }}</h1>
```

```
<p>Email: {{ auth()->user()->email }}</p>
```

```
<form method="POST" action="{{ route('logout') }}">
```

```
    @csrf
```

```
    <button type="submit">Logout</button>
```

```
</form>
```

```
@endsec
```

4. Form Validation

o Task: Create a contact form for users to submit their inquiries.

o Requirements:

- **Implement form validation to ensure all fields are filled out correctly.**
- **Display validation error messages using Blade.**

```
routes/web.php
```

```
use App\Http\Controllers\ContactController;

Route::get('/contact', [ContactController::class, 'index'])->name('contact.form');

Route::post('/contact', [ContactController::class, 'store'])->name('contact.submit');
```

Create Controller

```
php artisan make:controller ContactController
```

```
app/Http/Controllers/ContactController.php
```

```
use Illuminate\Http\Request;
```

```
class ContactController extends Controller
{
    public function index()
    {
        return view('contact');
    }

    public function store(Request $request)
    {
        // Validation
        $request->validate([
            'name' => 'required|min:3',
            'email' => 'required|email',
            'message' => 'required|min:10',
        ]);
        return back()->with('success', 'Your message has been sent!');
    }
}
```

```
resources/views/contact.blade.php
```

```
<!DOCTYPE html>
<html>
<head>
    <title>Contact Form</title>
</head>
<body>

<h2>Contact Us</h2>

<!-- Success Message -->
```

```

@if(session('success'))
    <p style="color: green;">{{ session('success') }}</p>
@endif

<form action="{{ route('contact.submit') }}" method="POST">
    @csrf

    <!-- Name -->
    <input type="text" name="name" placeholder="Enter Name" value="{{ old('name') }}">
    @error('name')
        <p style="color:red;">{{ $message }}</p>
    @enderror

    <br><br>

    <!-- Email -->
    <input type="email" name="email" placeholder="Enter Email" value="{{ old('email') }}">
    @error('email')
        <p style="color:red;">{{ $message }}</p>
    @enderror

    <br><br>

    <!-- Message -->
    <textarea name="message" placeholder="Enter Message">{{ old('message') }}</textarea>
    @error('message')
        <p style="color:red;">{{ $message }}</p>
    @enderror

    <br><br>

    <button type="submit">Submit</button>
</form>

</body>
</html>

```

5. RESTful API Development

o Task: Create a RESTful API for managing products in an inventory.

o Requirements:

- implement endpoints for creating, retrieving, updating, and deleting products.
- Use Laravel's resource controllers and API routes.
- Test the API using Postman.

routes/api.php

```
use App\Http\Controllers\Api\ProductController;
```

```
Route::apiResource('products', ProductController::class);
```

app/Http/Controllers/Api/ProductController.php

```
public function index()
```

```
{
```

```
    return Product::all();
```

```
}
```

```
public function store(Request $request)
```

```
{
```

```
    $request->validate([
```

```
        'name' => 'required',
```

```
        'price' => 'required|numeric',
```

```
        'quantity' => 'required|integer',
```

```
    ]);
```

```
    return Product::create($request->all());
```

```
}
```

```
public function show(Product $product)
```

```
{
```

```
    return $product;
```

```
}
```

```
public function update(Request $request, Product $product)
```

```

{
    $product->update($request->all());
    return $product;
}

public function destroy(Product $product)
{
    $product->delete();
    return response()->json(['message' => 'Product deleted']);
}

```

6. Using Laravel Middleware

o Task: Create a middleware that checks if a user is an admin.

o Requirements:

- Apply this middleware to certain routes to restrict access based on user roles.

Middleware Logic

app/Http/Middleware/AdminMiddleware.php

use Closure;

use Illuminate\Http\Request;

public function handle(Request \$request, Closure \$next)

```

{
    if (auth()->check() && auth()->user()->role === 'admin') {
        return $next($request);
    }

    return redirect('/')->with('error', 'Access denied. Admins only.');
```

app/Http/Kernel.php

```

protected $routeMiddleware = [
    'admin' => \App\Http\Middleware\AdminMiddleware::class,
];

```

Apply Middleware to Routes

```
Route::get('/admin/dashboard', function () {  
    return "Admin Dashboard";  
})->middleware('admin');
```

7. Laravel Notifications

o Task: Implement a notification system that sends email notifications when a new blog post is published.

o Requirements:

- Use Laravel's notification system to send notifications.
- Set up a queue for processing notifications.

Create Notification

```
php artisan make:notification NewPostNotification
```

```
app/Notifications/NewPostNotification.php
```

```
use Illuminate\Notifications\Notification;
```

```
use Illuminate\Notifications\Messages\MailMessage;
```

```
class NewPostNotification extends Notification
```

```
{
```

```
    protected $post;
```

```
    public function __construct($post)
```

```
    {
```

```
        $this->post = $post;
```

```
    }
```

```
    public function via($notifiable)
```

```
    {
```

```
        return ['mail']; // send via email
```

```
}
```

```
public function toMail($notifiable)
```

```
{
```

```
    return (new MailMessage)
```

```
        ->subject('New Blog Post Published')
```

```
        ->greeting('Hello!')
```

```
        ->line('A new blog post has been published.')
```

```
        ->line('Title: ' . $this->post->title)
```

```
        ->action('View Post', url('/posts/' . $this->post->id))
```

```
        ->line('Thank you!');
```

```
}
```

```
}
```

```
app/Models/User.php
```

```
use Illuminate\Notifications\Notifiable;
```

```
class User extends Authenticatable
```

```
{
```

```
    use Notifiable;
```

```
}
```

```
PostController.php
```

```
use App\Models\Post;
```

```
use App\Models\User;
```

```
use App\Notifications\NewPostNotification;
```

```
public function store(Request $request)
```

```
{
```

```
    $post = Post::create($request->all());
```

```
    // Send notification to all users
```

```
    $users = User::all();
```

```
    foreach ($users as $user) {
```

```
$user->notify(new NewPostNotification($post));  
}
```

```
return redirect()->route('posts.index');  
}
```

8. Integrating a Payment Gateway

o Task: Integrate a payment gateway (e.g., Stripe or PayPal) into your blog application.

o Requirements:

- Create a feature that allows users to make donations or purchases.
- Handle payment processing and success/error responses.

9. Building a CRUD Application with Resource Controllers

o Task: Create a resource controller for managing categories in the blog application.

o Requirements:

- Implement all CRUD operations using resource routes.
- Create views for adding and editing categories.

Create Resource Controller

```
php artisan make:controller CategoryController --resource
```

```
routes/web.php
```

```
use App\Http\Controllers\CategoryController;
```

```
Route::resource('categories', CategoryController::class);
```

```
app/Http/Controllers/CategoryController.php
```

```
public function index()  
{  
    $categories = Category::all();  
    return view('categories.index', compact('categories'));  
}
```

```
public function create()  
{
```

```
        return view('categories.create');
    }

    public function store(Request $request)
    {
        $request->validate([
            'name' => 'required'
        ]);

        Category::create($request->all());
        return redirect()->route('categories.index');
    }

    public function edit(Category $category)
    {
        return view('categories.edit', compact('category'));
    }

    public function update(Request $request, Category $category)
    {
        $request->validate([
            'name' => 'required'
        ]);

        $category->update($request->all());
        return redirect()->route('categories.index');
    }

    public function destroy(Category $category)
    {
        $category->delete();
        return redirect()->route('categories.index');
```

```
}
```

```
index.blade.php
```

```
@extends('layout')
```

```
@section('content')
```

```
<a href="{{ route('categories.create') }}">Add Category</a>
```

```
@foreach($categories as $category)
```

```
<p>{{ $category->name }}</p>
```

```
<a href="{{ route('categories.edit', $category->id) }}">Edit</a>
```

```
<form action="{{ route('categories.destroy', $category->id) }}" method="POST">
```

```
@csrf
```

```
@method('DELETE')
```

```
<button type="submit">Delete</button>
```

```
</form>
```

```
@endforeach
```

```
@endsection
```

```
create.blade.php
```

```
@extends('layout')
```

```
@section('content')
```

```
<form action="{{ route('categories.store') }}" method="POST">
```

```
@csrf
```

```
<input type="text" name="name" placeholder="Category Name">
```

```
<button type="submit">Save</button>
```

```
</form>
```

```
@endsection
```

```
edit.blade.php
```

```
@extends('layout')
```

```
@section('content')
```

```
<form action="{{ route('categories.update', $category->id) }}" method="POST">
```

```
    @csrf
```

```
    @method('PUT')
```

```
    <input type="text" name="name" value="{{ $category->name }}">
```

```
    <button type="submit">Update</button>
```

```
</form>
```

```
@endsection
```


